

Linked List: 100 Practice Problems

Solve every problem with your own user-defined functions. No built-in list classes.

Ground rules

- Write your own Node and list functions. Do not use LinkedList, ArrayList, list, or any collection framework class.
- Arrays or hash sets may be used only where a problem says so, or when you are checking your answer.
- For each problem, test: empty list, one node, two nodes, and a normal case.
- Prefer changing links over changing data whenever the problem is about rearranging nodes.
- Attempt each problem first without notes. Tick the box only when your code passes all your tests.

Starter node (pseudocode, use any language)

```
Node:  
  data  
  next -> Node or NULL  
  
head -> [10] -> [20] -> [30] -> NULL
```

Levels: Easy (42) Medium (41) Hard (17)

Contents

- A. Basics: Creation and Traversal (problems 1-10)
- B. Insertion (problems 11-20)
- C. Deletion (problems 21-30)
- D. Search and Query (problems 31-40)
- E. Modification and Utility (problems 41-50)
- F. Reversal and Rearrangement (problems 51-60)
- G. Two-Pointer and Classic Techniques (problems 61-70)
- H. Merge, Sort and Split (problems 71-80)
- I. Doubly Linked List (problems 81-88)
- J. Circular Linked List (problems 89-94)
- K. Advanced and Interview Problems (problems 95-100)

A. Basics: Creation and Traversal

#	Problem	Level	Done
1	Define the Node Define a Node structure/class holding data and a next reference. Write createNode(value) that returns a new node.	Easy	[]
2	Build a list from an array Write buildList(arr) that creates a linked list from an array and returns the head.	Easy	[]
3	Print the list Traverse and print in the format 1 -> 2 -> 3 -> NULL. Handle an empty list.	Easy	[]
4	Count nodes (iterative) Return the number of nodes using a loop.	Easy	[]
5	Count nodes (recursive) Return the number of nodes using recursion only.	Easy	[]
6	Sum of all elements Return the sum of all node values.	Easy	[]
7	Maximum element Return the largest value in the list.	Easy	[]
8	Minimum element Return the smallest value in the list.	Easy	[]
9	Print recursively Print the list in forward order using recursion.	Easy	[]
10	Print in reverse (no modification) Print elements in reverse order using recursion, without changing any links.	Easy	[]

B. Insertion

#	Problem	Level	Done
11	Insert at head Insert a new node at the beginning and return the new head.	Easy	[]
12	Insert at tail (with traversal) Insert a new node at the end by walking to the last node.	Easy	[]
13	Insert at tail in O(1) Maintain a tail pointer so that insertion at the end needs no traversal.	Easy	[]
14	Insert at position Insert at a given 0-based index. Handle index 0, index equal to length, and invalid indexes.	Easy	[]
15	Insert after a given value Insert a new node right after the first node containing value x.	Easy	[]
16	Insert before a given value Insert a new node right before the first node containing value x (watch the head case).	Medium	[]

#	Problem	Level	Done
17	Insert into a sorted list Insert a value into an ascending sorted list so that it stays sorted.	Medium	[]
18	Insert at the middle Insert a node at the middle position of the list (define your rule for even lengths).	Medium	[]
19	Insert at kth position from the end Insert a node so that it becomes the kth node counted from the end.	Medium	[]
20	Insert only if unique Insert a value only if it is not already present in the list.	Easy	[]

C. Deletion

#	Problem	Level	Done
21	Delete the head Remove the first node and return the new head.	Easy	[]
22	Delete the tail Remove the last node. Handle single-node and empty lists.	Easy	[]
23	Delete at position Delete the node at a given 0-based index. Handle invalid indexes.	Easy	[]
24	Delete first occurrence of a value Remove the first node whose data equals x.	Easy	[]
25	Delete all occurrences of a value Remove every node whose data equals x, including consecutive ones at the head.	Medium	[]
26	Delete the node after a given node Given a pointer to a node, delete the node that follows it.	Easy	[]
27	Delete the middle node Remove the middle node of the list in a single pass.	Medium	[]
28	Delete kth node from the end Remove the kth node from the end in one pass using two pointers.	Medium	[]
29	Delete the entire list Free/clear every node and leave the list empty (head = NULL).	Easy	[]
30	Delete a node given only its pointer You are given only a pointer to a node (not the head, not the tail). Delete it from the list.	Medium	[]

D. Search and Query

#	Problem	Level	Done
31	Search for a value Return true/false depending on whether x exists in the list.	Easy	[]

#	Problem	Level	Done
32	Index of first occurrence Return the 0-based index of x, or -1 if it is not found.	Easy	[]
33	All indexes of a value Return every index at which x occurs.	Easy	[]
34	Get element at index Return the value at a given index, with proper handling of out-of-range input.	Easy	[]
35	kth node from the end Return the kth node from the end using (a) length calculation and (b) two pointers.	Medium	[]
36	Find the middle element Return the middle value using the slow/fast pointer technique.	Easy	[]
37	Count occurrences Return how many times x appears in the list.	Easy	[]
38	Second largest element Return the second largest distinct value in a single traversal.	Medium	[]
39	Check if sorted Return true if the list is sorted in non-decreasing order.	Easy	[]
40	Longest run of equal values Return the length of the longest streak of consecutive equal values.	Medium	[]

E. Modification and Utility

#	Problem	Level	Done
41	Update value at index Change the data of the node at a given index.	Easy	[]
42	Replace all occurrences Replace every x with y in the list.	Easy	[]
43	Copy the list Create a deep copy (new nodes, same values) of the entire list.	Medium	[]
44	Convert list to array Return an array containing all values in order.	Easy	[]
45	Append one list to another Attach list B to the end of list A and return the head.	Easy	[]
46	Swap two nodes by links Swap the nodes holding values x and y by changing links, not data. Handle adjacent and head cases.	Hard	[]
47	Swap first and last data Swap the data of the first and last nodes.	Easy	[]
48	Rotate left by k Rotate the list to the left by k positions (k may exceed length).	Medium	[]

#	Problem	Level	Done
49	Rotate right by k Rotate the list to the right by k positions.	Medium	[]
50	Compare two lists Return true if two lists have identical values in identical order (iterative and recursive).	Easy	[]

F. Reversal and Rearrangement

#	Problem	Level	Done
51	Reverse iteratively Reverse the list using prev, curr and next pointers.	Easy	[]
52	Reverse recursively Reverse the list using recursion.	Medium	[]
53	Reverse the first k nodes Reverse only the first k nodes and keep the rest attached.	Medium	[]
54	Reverse in groups of k Reverse every consecutive group of k nodes. Leftover nodes stay as they are.	Hard	[]
55	Reverse between positions m and n Reverse only the portion from position m to n in one pass.	Hard	[]
56	Reverse alternate k groups Reverse the first k nodes, skip the next k, reverse the next k, and so on.	Hard	[]
57	Swap nodes in pairs Swap every two adjacent nodes by changing links (1->2->3->4 becomes 2->1->4->3).	Medium	[]
58	Move last node to front Take the last node and make it the new head.	Easy	[]
59	Segregate even and odd values Rearrange so all even values come first, then odd values, keeping relative order.	Medium	[]
60	Group by index parity Place all nodes at odd indexes first, followed by nodes at even indexes, in place.	Medium	[]

G. Two-Pointer and Classic Techniques

#	Problem	Level	Done
61	Detect a cycle Use Floyd's slow/fast pointers to detect whether a loop exists.	Medium	[]
62	Find the start of the cycle Return the node where the cycle begins.	Hard	[]
63	Length of the cycle Return the number of nodes in the loop.	Medium	[]

#	Problem	Level	Done
64	Remove the cycle Break the loop so that the list ends in NULL.	Hard	[]
65	Palindrome check Check if the list reads the same forward and backward in O(1) extra space.	Medium	[]
66	Intersection point of two lists Two lists merge into one at some node. Find that node.	Medium	[]
67	Remove duplicates from sorted list Delete repeated values so each value appears once.	Easy	[]
68	Remove duplicates from unsorted list Delete repeated values from an unsorted list (first with nested loops, then with a hash set).	Medium	[]
69	Keep only distinct values Remove every value that appears more than once (delete all copies, not just extras) from a sorted list.	Medium	[]
70	Reorder list Rearrange L0, L1, ..., Ln into L0, Ln, L1, Ln-1, L2, Ln-2, ... in place.	Hard	[]

H. Merge, Sort and Split

#	Problem	Level	Done
71	Merge two sorted lists Merge two ascending lists into one sorted list by relinking nodes.	Medium	[]
72	Merge k sorted lists Merge k sorted lists into one (pair-wise merging or divide and conquer).	Hard	[]
73	Bubble sort on a list Sort by swapping node data, then again by swapping links.	Medium	[]
74	Insertion sort on a list Sort the list by repeatedly inserting each node into a growing sorted list.	Medium	[]
75	Selection sort on a list Sort the list by repeatedly selecting the minimum node.	Medium	[]
76	Merge sort on a list Sort in O(n log n) using split-middle, recursive sort and merge.	Hard	[]
77	Partition around a value Rearrange so that nodes smaller than x come before nodes greater or equal to x, keeping relative order.	Medium	[]
78	Split into two halves Split the list into a front half and a back half (front gets the extra node when odd).	Easy	[]
79	Alternating split Split the list into two lists by taking nodes alternately (1st, 3rd, ... and 2nd, 4th, ...).	Medium	[]

#	Problem	Level	Done
80	Common elements of two sorted lists Build a new list containing values present in both sorted lists.	Medium	[]

I. Doubly Linked List

#	Problem	Level	Done
81	Create and print both ways Define a doubly linked node. Build a list and print it forward and backward.	Easy	[]
82	Insert at head and tail Insert at either end with all prev and next links correct.	Easy	[]
83	Insert at position Insert at a given index in a doubly linked list.	Medium	[]
84	Delete head, tail or position Write three delete functions and handle single-node lists safely.	Medium	[]
85	Delete by value Delete the first node with value x, fixing both links of its neighbours.	Medium	[]
86	Reverse a doubly linked list Reverse the list by swapping prev and next of each node.	Medium	[]
87	Pairs with a given sum In a sorted doubly linked list, find all pairs whose sum equals x using two pointers.	Hard	[]
88	Remove duplicates from sorted DLL Delete duplicate values from a sorted doubly linked list.	Medium	[]

J. Circular Linked List

#	Problem	Level	Done
89	Create and print Build a circular singly linked list and print it exactly once around.	Easy	[]
90	Insert at beginning and end in O(1) Keep only a tail pointer and insert at both ends in constant time.	Medium	[]
91	Delete a node Delete by value from a circular list, including the head/tail cases.	Medium	[]
92	Split into two halves Split a circular list into two circular lists of half size each.	Hard	[]
93	Josephus problem n people stand in a circle and every kth person is eliminated. Find the survivor using your circular list.	Hard	[]
94	Check if a list is circular Given the head of a list, check whether it is circular (returns to the head) or NULL-terminated.	Medium	[]

K. Advanced and Interview Problems

#	Problem	Level	Done
95	Add two numbers (digits reversed) Each list stores a number with the least significant digit first. Return their sum as a list.	Medium	[]
96	Add two numbers (digits forward) Each list stores a number with the most significant digit first. Add them without reversing the input lists.	Hard	[]
97	Multiply two numbers Multiply two numbers represented as lists and return the product as a list.	Hard	[]
98	Flatten a multilevel list Each node may have a child list. Flatten everything into a single-level list.	Hard	[]
99	Clone a list with random pointers Each node has next and random pointers. Create a deep copy in $O(n)$.	Hard	[]
100	Design an LRU cache Build an LRU cache with get and put in $O(1)$ using a doubly linked list plus a hash map.	Hard	[]

When you finish

Re-solve a handful of problems using the collection framework's `LinkedList` and compare what the framework does for you. Then move on to stacks and queues, which you can build on top of your own linked list.